



ADVANCED SECURE E-COMMERCE MANAGEMENT SYSTEM

DATABASE PROGRAMMING WITH ORACLE DATABASE

Prepared By:
Elzubair

Student ID:
31363_2025

Project Name:
31363_2025_Elzubair_ECommerce_DB

Year:2026

Introduction:

Advanced Secure E-Commerce Management System is a database project developed using Oracle SQL and PL/SQL.

The system is designed to manage important e-commerce operations including products, employees, customers, sales transactions, and security monitoring.

This project demonstrates database design, implementation, transaction management, and PL/SQL programming techniques to build a secure and efficient database system.

Problem Statement:

Many e-commerce systems face challenges in managing large amounts of data, maintaining data integrity, and ensuring secure transactions.

The main problems include:

- Difficulty in managing products, customers, employees, and sales information.
- Lack of proper monitoring of database activities.
- Risk of unauthorized or restricted transactions.
- Need for automated business rules and security controls.

This project provides a secure database solution to manage e-commerce operations efficiently using Oracle SQL and PL/SQL.

Objectives:

The main objectives of this project are:

- To design and implement an E-Commerce database system using Oracle Database.
- To manage products, customers, employees, and sales transactions efficiently.
- To apply SQL and PL/SQL programming concepts in database operations.
- To implement triggers, procedures, functions, and packages for automation.
- To improve database security using auditing and business rules.

- To maintain data integrity using constraints and transaction control.

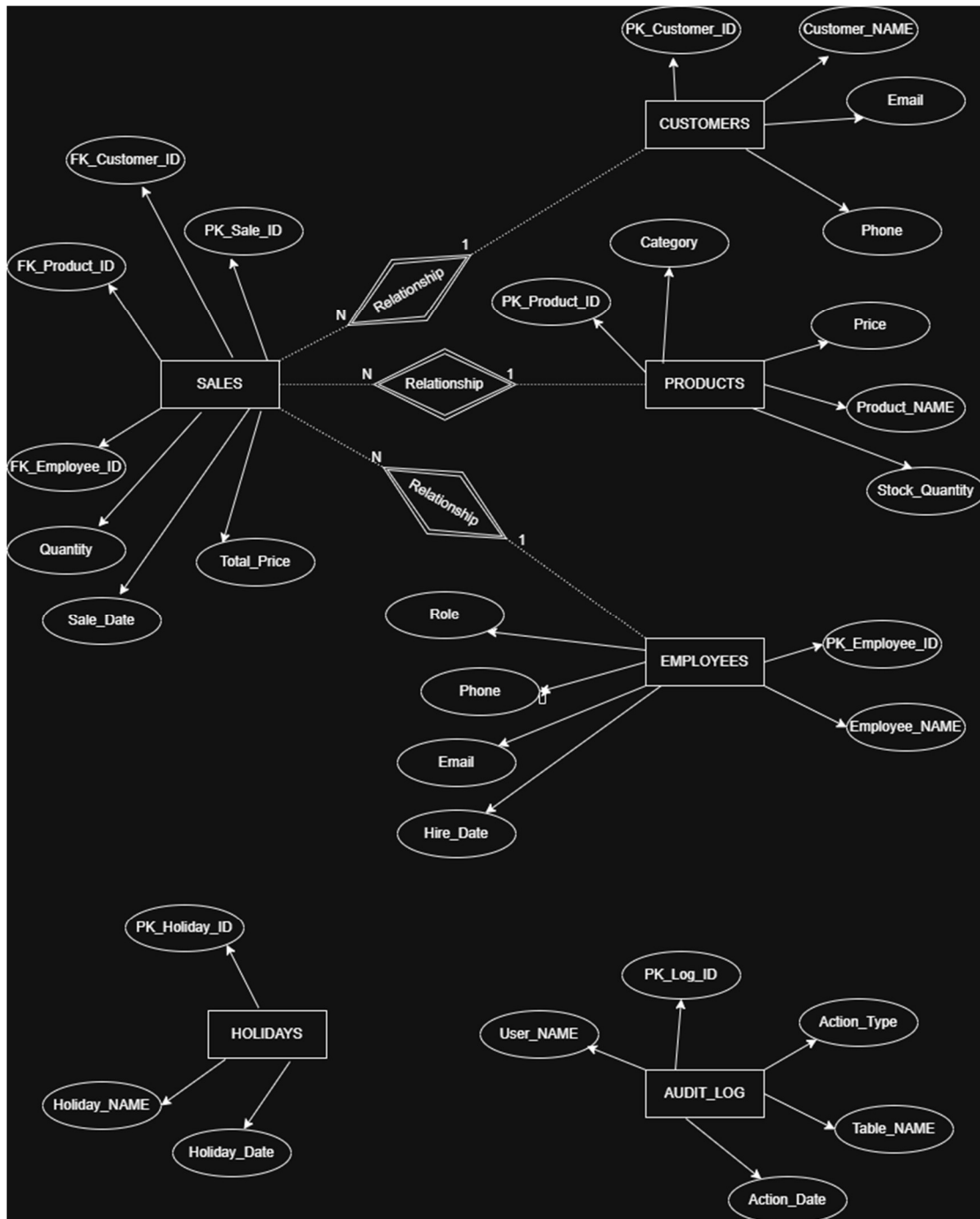
Database Design:

The database was designed to support the main operations of the Advanced Secure E-Commerce Management System.

The system consists of several related tables:

- Employees: Stores employee information responsible for managing operations.
- Products: Stores product details such as product name, price, and stock quantity.
- Customers: Stores customer information.
- Sales: Records sales transactions and connects customers, products, and employees.
- Holidays: Stores public holidays used for business rule validation.
- Audit_Log: Stores database activities for security monitoring.

Primary Keys and Foreign Keys are used to establish relationships between tables and maintain data integrity.



Database Implementation:

The system database was implemented using Oracle Database with SQL and PL/SQL programming.

Tables:

The main tables including Products, Customers, Employees, Sales, Holidays, and Audit_Log were created to store and manage system data.

Constraints:

Primary Keys, Foreign Keys, and validation rules were applied to maintain data integrity and ensure correct relationships between tables.

Data Insertion:

Sample data was inserted into the tables to test database operations.

Transaction Control:

COMMIT and ROLLBACK operations were implemented to manage database transactions safely.

Security Implementation:

Audit logging and business rules were applied to monitor activities and restrict unauthorized operations.

```
SQL> CREATE TABLE Customers (  
2     customer_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
3     customer_name VARCHAR2(100) NOT NULL,  
4     email VARCHAR2(100),  
5     phone VARCHAR2(20)  
6 );  
  
Table created.
```

```
SQL> INSERT INTO Employees (employee_name, email, role)  
2 VALUES ('Ahmed Ali', 'ahmed@gmail.com', 'Sales Manager');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Employees (employee_name, email, role)  
2 VALUES ('Sara Mohamed', 'sara@gmail.com', 'Cashier');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Employees (employee_name, email, role)  
2 VALUES ('John Peter', 'john@gmail.com', 'Sales Officer');  
  
1 row created.  
  
SQL>
```

PL/SQL Implementation:

PL/SQL was used to automate database operations and implement advanced database programming features.

The following PL/SQL components were implemented:

Triggers:

Used to enforce business rules, control transactions, and record database activities.

Procedures:

Used to perform specific database operations such as displaying and processing data.

Functions:

values.

Packages:

Used to organize related procedures and functions into a single program unit.

Cursors:

Used for processing multiple rows returned from database queries.

Exception Handling:

Implemented to handle errors and ensure safe execution of PL/SQL programs.

```
SQL> CREATE OR REPLACE PROCEDURE show_products
2 IS
3     CURSOR c_products IS
4         SELECT product_name, price, stock_quantity
5           FROM Products;
6
7     v_name Products.product_name%TYPE;
8     v_price Products.price%TYPE;
9     v_stock Products.stock_quantity%TYPE;
10
11 BEGIN
12     FOR rec IN c_products LOOP
13         DBMS_OUTPUT.PUT_LINE(
14             rec.product_name || ' - Price: ' ||
15             rec.price || ' - Stock: ' ||
16             rec.stock_quantity
17         );
18     END LOOP;
19 END;
20 /

Procedure created.

SQL> SET SERVEROUTPUT ON;
SQL> EXEC show_products;
Laptop - Price: 800 - Stock: 13
Smartphone - Price: 500 - Stock: 20
Headphones - Price: 50 - Stock: 30
Keyboard - Price: 30 - Stock: 10
Mouse - Price: 20 - Stock: 15

PL/SQL procedure successfully completed.
```



```
SQL Plus
9      SELECT COUNT(*)
10     INTO v_count
11     FROM Holidays
12     WHERE holiday_date = TRUNC(SYSDATE);
13
14     IF v_count > 0 THEN
15         RAISE_APPLICATION_ERROR(
16             -20001,
17             'Sales are not allowed on public holidays'
18         );
19     END IF;
20
21 END;
22 /

Trigger created.

SQL> CREATE OR REPLACE TRIGGER trg_block_holiday_sales
2 BEFORE INSERT ON Sales
3 FOR EACH ROW
4
5 DECLARE
6     v_count NUMBER;
7
8 BEGIN
9     SELECT COUNT(*)
10    INTO v_count
11    FROM Holidays
12    WHERE holiday_date = TRUNC(SYSDATE);
13
14    IF v_count > 0 THEN
15        RAISE_APPLICATION_ERROR(
16            -20001,
17            'Sales are not allowed on public holidays'
18        );
19    END IF;
20
21 END;
22 /

Trigger created.

SQL>

SQL> CREATE OR REPLACE TRIGGER trg_compound_products
2 FOR INSERT OR UPDATE ON Products
3 COMPOUND TRIGGER
4
5 BEFORE STATEMENT IS
6 BEGIN
7     DBMS_OUTPUT.PUT_LINE('Starting Product Operation...');
8 END BEFORE STATEMENT;
9
10 AFTER STATEMENT IS
11 BEGIN
12     DBMS_OUTPUT.PUT_LINE('Product Operation Completed.');
```

Security and Business Rules:

The system includes security features and business rules to improve database protection and reliability.

Security Features:

- Audit_Log is used to record important database activities.
- User activities are tracked for monitoring purposes.
- Database operations can be controlled according to system rules.

Business Rules:

- INSERT, UPDATE, and DELETE operations can be restricted during specific periods.
- Public holidays stored in the Holidays table are used for validation.
- Triggers are used to automatically enforce these restrictions.

These security mechanisms help maintain data consistency and prevent unauthorized database changes.

Results:

The system was successfully implemented and tested using Oracle SQL and PL/SQL.

The main results achieved are:

- Successfully created the E-Commerce database structure.
- Implemented tables, relationships, and constraints.
- Inserted and managed system data successfully.
- Successfully executed SQL and PL/SQL programs including triggers, procedures, functions, and packages.
- Implemented security features through Audit_Log and business rules.
- Tested transaction operations using COMMIT and ROLLBACK.

Screenshots of the implementation and testing results are included in this report.

```

4      price NUMBER(10,2) NOT NULL,
5      stock_quantity NUMBER DEFAULT 0
6  );

Table created.

SQL>
SQL> CREATE TABLE Sales (
2      sale_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
3      product_id NUMBER NOT NULL,
4      employee_id NUMBER NOT NULL,
5      customer_id NUMBER,
6      quantity NUMBER NOT NULL,
7      sale_date DATE DEFAULT SYSDATE,
8      total_amount NUMBER(10,2) NOT NULL,
9      CONSTRAINT fk_sales_product FOREIGN KEY (product_id) REFERENCES Products(product_id),
10     CONSTRAINT fk_sales_employee FOREIGN KEY (employee_id) REFERENCES Employees(employee_id)
11 );

Table created.

SQL>
SQL> CREATE TABLE Holidays (
2      holiday_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
3      holiday_date DATE NOT NULL UNIQUE,
4      holiday_name VARCHAR2(100) NOT NULL
5  );

Table created.

SQL>
SQL> CREATE TABLE Audit_Log (
2      log_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
3      action_type VARCHAR2(50) NOT NULL,
4      table_name VARCHAR2(50) NOT NULL,
5      action_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
6      user_name VARCHAR2(50),
7      details VARCHAR2(4000)
8  );

Table created.

SQL>

```

```

10      -- Check weekday
11      v_day := TO_CHAR(SYSDATE, 'DAY');
12
13      IF TRIM(v_day) IN ('MONDAY','TUESDAY','WEDNESDAY','THURSDAY','FRIDAY') THEN
14          RAISE_APPLICATION_ERROR(
15              -20001,
16              'Transactions are not allowed on weekdays'
17          );
18      END IF;
19
20      -- Check public holidays
21      SELECT COUNT(*)
22      INTO v_holiday
23      FROM Holidays
24      WHERE holiday_date = TRUNC(SYSDATE);
25
26      IF v_holiday > 0 THEN
27          RAISE_APPLICATION_ERROR(
28              -20002,
29              'Transactions are not allowed on public holidays'
30          );
31      END IF;
32
33  END;
34  /

Trigger created.

SQL> INSERT INTO Sales
2  (product_id, employee_id, customer_id, quantity, total_amount)
3  VALUES
4  (1, 1, 1, 1, 800);
INSERT INTO Sales
*
ERROR at line 1:
ORA-20001: Transactions are not allowed on weekdays
ORA-06512: at "31363_2025_Elzubair_ECommerce.DB.TRG_BLOCK_BUSINESS_DAYS", line
10
ORA-04088: error during execution of trigger
'31363_2025_Elzubair_ECommerce.DB.TRG_BLOCK_BUSINESS_DAYS'

SQL>

```

```

SQL> INSERT INTO Products (product_name, price, stock_quantity)
2  VALUES ('Monitor', 250, 5);

1 row created.

SQL> ROLLBACK;

Rollback complete.

SQL> SELECT * FROM Products
2  WHERE product_name = 'Monitor';

no rows selected

SQL>

```

Conclusion:

The Advanced Secure E-Commerce Management System was successfully developed using Oracle SQL and PL/SQL.

The project achieved its objectives by designing a complete database system, implementing relationships between tables, and applying database programming techniques.

The use of triggers, procedures, functions, packages, audit mechanisms, and transaction control improved system security, automation, and data integrity.

This project demonstrates the importance of database management systems in building reliable and secure e-commerce applications.